Gradient Descent for Logistic Regression

Syed Hamed Raza

University of Management and Technology (UMT), Lahore
Course: Deep Learning and Neural Network
Course Code: CS4152

November 6, 2025

Recap

- Model: $\hat{y} = \sigma(W^T x + b)$ with $\sigma(z) = \frac{1}{1+e^{-z}}$.
- Cost function (binary cross-entropy):

$$J(W, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right].$$

- Goal: find (W, b) that minimize $J(W, b)$.

What is Gradient Descent?

- Iterative optimization to minimize a cost $J(W, b)$.
- Update rules:

$$W \leftarrow W - \alpha \frac{\partial J}{\partial W}, \qquad b \leftarrow b - \alpha \frac{\partial J}{\partial b}.$$

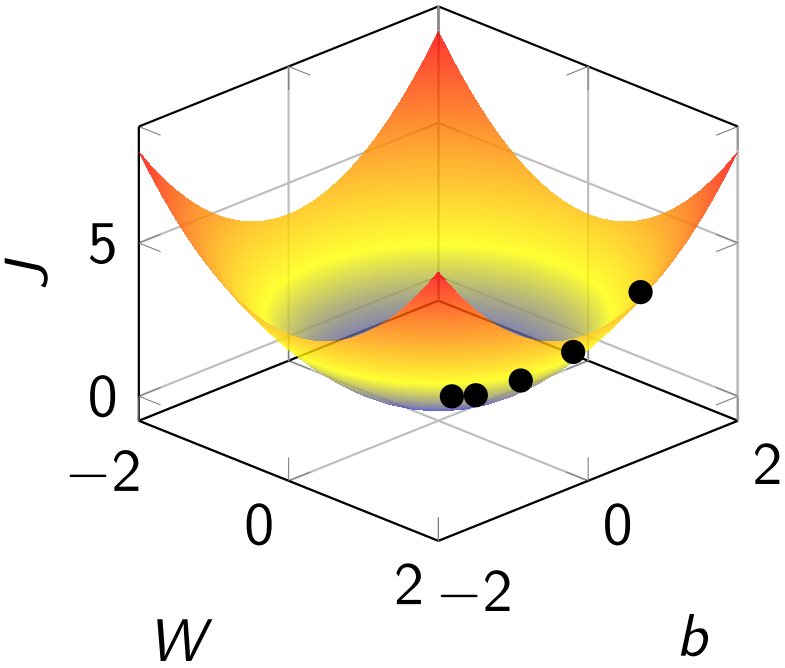
- α is the **learning rate** (step size).
- Moves parameters along the direction of **steepest descent**.

Convexity of Logistic Regression

- For logistic regression, $J(W, b)$ is **convex**.
- Implies a **single global minimum**.
- Initialization (zeros or small random) converges to the same optimum.

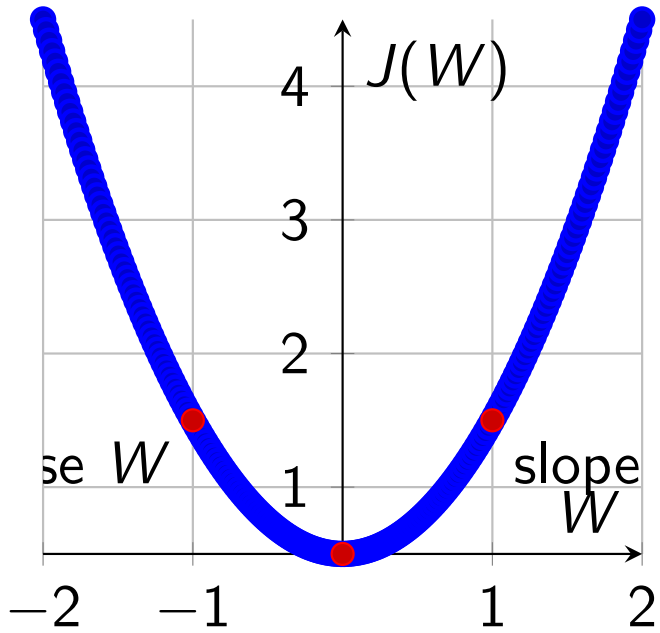
Gradient Descent Intuition

- Start at an initial (W, b).
- Take steps downhill; each step lowers J.
- Stop when improvement becomes negligible (convergence).



1D View: Sign of the Derivative

- Derivative $\frac{dJ}{dW}$ is the **slope** at current W.
- If $\frac{dJ}{dW} > 0 \Rightarrow$ step left (decrease W).
- If $\frac{dJ}{dW} < 0 \Rightarrow$ step right (increase W).
- Converge when slope ≈ 0.



Update Rules Recap

$$W \leftarrow W - \alpha \frac{\partial J(W, b)}{\partial W},$$
$$b \leftarrow b - \alpha \frac{\partial J(W, b)}{\partial b}.$$

- $\frac{\partial J}{\partial W}$ and $\frac{\partial J}{\partial b}$ are the gradients used in updates.
- Negative sign moves parameters toward lower cost.

Implementation Notes

- Code convention: `dW` for $\frac{\partial J}{\partial W}$, `db` for $\frac{\partial J}{\partial b}$.
- Vectorized Python-style updates:

```
W = W - alpha * dW
b = b - alpha * db
```

- Repeat until convergence (or max iterations / small change in J).

Derivative vs Partial Derivative

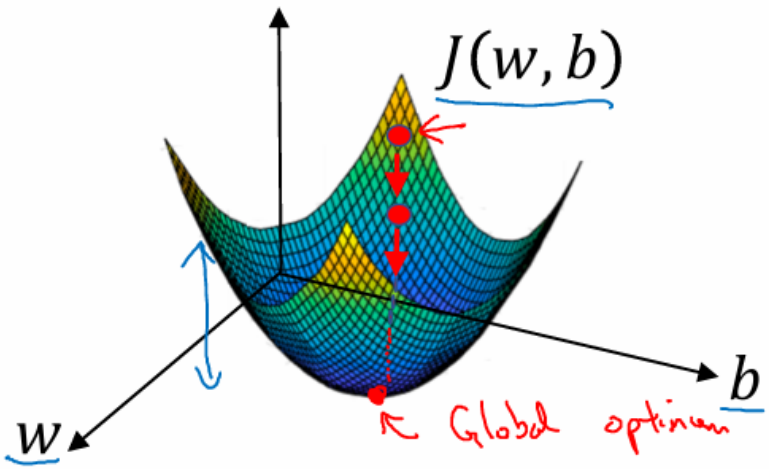
- Single variable: use $\mathrm{d}J/\mathrm{d}W$.
- Multiple variables: use partials $\partial J/\partial W$, $\partial J/\partial b$.
- Both quantify the local **rate of change** (slope) of J.

Gradient Descent

Recap: $\hat{y} = \sigma(w^T x + b)$, $\sigma(z) = \frac{1}{1+e^{-z}}$

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

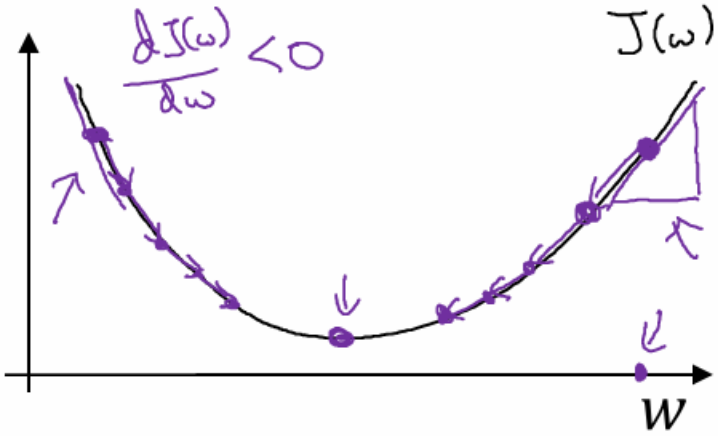
Want to find w, b that minimize $J(w,b)$



Andrew Ng

Gradient Descent 2

Gradient Descent



Repeat {

$w := w - \alpha \frac{dJ(w)}{dw}$

learning rate

"dw"

}

$w := w - \alpha dw$

$\frac{dJ(w)}{dw} = ?$

$J(w,b)$

$w := w - \alpha \frac{\partial J(w,b)}{\partial w}$

$b := b - \alpha \frac{\partial J(w,b)}{\partial b}$

$\frac{\partial J(w,b)}{\partial w}$ → dw

$\frac{\partial J(w,b)}{\partial b}$ → db

∂ "partial derivative"

J

Andrew Ng

Summary

- Gradient descent iteratively minimizes $J(W, b)$ by stepping opposite the gradient.
- Logistic regression's convex cost ensures a unique optimum.
- Foundations here extend directly to training deep neural networks.